



**Università  
di Genova**

**DIBRIS** DIPARTIMENTO  
DI INFORMATICA, BIOINGEGNERIA,  
ROBOTICA E INGEGNERIA DEI SISTEMI

# High Performance Ecoinformatics

Enrico Pezzano

Master Thesis

Università di Genova, DIBRIS Via Opera Pia, 13 16145 Genova, Italy  
<https://www.dibris.unige.it/>



**MSc Computer Science**  
Data Science and Engineering Curriculum

# High Performance Econinformatics

Enrico Pezzano

Advisor: Prof. Daniele D'Agostino (DIBRIS)  
Prof. Gabriele Casazza (DISTAV)

Examiner: Prof. Matteo Dell'Amico

October, 2026



## *DEDICATION*



**Abstract** Ecoinformatics is a modern method of doing science that enables scientists to generate new knowledge through innovative tools and approaches. Ecology is increasingly becoming a data-intensive science, involving the acquisition, preservation, and analysis of vast amounts of data from a variety of sources, including satellite and airborne remote sensing, instruments, sensors, and human observations. Ecologists are increasingly addressing larger-scale questions of both scientific and societal relevance, such as climate change and its consequences for ecosystems and related ecosystem services in the future. The topic of this thesis project stems from a research study conducted by the Department of Earth, Environmental, and Life Sciences (DISTAV, University of Genoa) that analyzes the shift in the distribution of natural habitats in the Alps due to future climate change to support decision-making processes regarding conservation planning and prioritization. The analysis is conducted using Species Distribution Models (SDMs), a dataset of approximately 3 million data points, and future climate change scenarios based on General Circulation Models (climate models). The statistical software R (<https://www.r-project.org/>) is used to generate future predictions of habitat distribution changes and biodiversity loss. The thesis candidate will work on improving the script and algorithmic functions, written in R, to streamline model analysis based on their knowledge and computer science background. This thesis project leverages interdisciplinary collaboration between DIBRIS and DISTAV to achieve better scientific results.



# Table of Contents

|                  |                                                                        |           |
|------------------|------------------------------------------------------------------------|-----------|
| <b>Chapter 1</b> | <b>Introduction and Motivation</b>                                     | <b>10</b> |
| 1.1              | Project context (taken from <code>Methods_GC_LD.pdf</code> ) . . . . . | 10        |
| 1.2              | Thesis motivation . . . . .                                            | 11        |
| 1.3              | Objectives and contribution . . . . .                                  | 12        |
| 1.4              | Preliminary evaluation method . . . . .                                | 12        |
| <b>Chapter 2</b> | <b>Background</b>                                                      | <b>14</b> |
| 2.1              | Ecoinformatics . . . . .                                               | 14        |
| 2.2              | Biological study context . . . . .                                     | 14        |
| 2.3              | Data used . . . . .                                                    | 15        |
| 2.4              | Related work . . . . .                                                 | 15        |
| <b>Chapter 3</b> | <b>Code as is</b>                                                      | <b>16</b> |
| 3.1              | R . . . . .                                                            | 16        |
| 3.2              | BIOMOD2 . . . . .                                                      | 16        |
| 3.3              | Code pipeline . . . . .                                                | 17        |
| 3.4              | Original execution times . . . . .                                     | 17        |
| 3.5              | Improved execution times . . . . .                                     | 17        |
| 3.6              | Preliminary measurements on Spartaco . . . . .                         | 18        |
| <b>Chapter 4</b> | <b>My new code</b>                                                     | <b>19</b> |

|                  |                                                   |           |
|------------------|---------------------------------------------------|-----------|
| 4.1              | Container . . . . .                               | 19        |
| 4.2              | Parallel model execution . . . . .                | 19        |
| 4.3              | Parallel I/O . . . . .                            | 20        |
| <b>Chapter 5</b> | <b>Experimental results</b>                       | <b>21</b> |
| 5.1              | Leonardo . . . . .                                | 21        |
| 5.2              | Tests . . . . .                                   | 22        |
| 5.3              | Discussion and lessons learned . . . . .          | 23        |
| 5.4              | Operational difficulties on Leonardo . . . . .    | 24        |
| 5.4.1            | Projection memory and process forks . . . . .     | 24        |
| 5.4.2            | Concurrent writes and stale output . . . . .      | 24        |
| 5.4.3            | Long phases without visible progress . . . . .    | 25        |
| 5.4.4            | Scheduling, maintenance and time limits . . . . . | 25        |
| 5.4.5            | Accounting and budget exhaustion . . . . .        | 25        |
| 5.4.6            | Completion and scientific validation . . . . .    | 26        |
| <b>Chapter 6</b> | <b>Conclusions and future development</b>         | <b>27</b> |
| 6.1              | Conclusions . . . . .                             | 27        |
| 6.2              | Future development . . . . .                      | 27        |
| 6.3              | Ecoinformatics . . . . .                          | 27        |
| 6.4              | Data used . . . . .                               | 27        |
| 6.5              | Computer science objective . . . . .              | 27        |
| 6.6              | Interdisciplinary collaboration . . . . .         | 28        |
| <b>Chapter 7</b> | <b>Appendix</b>                                   | <b>29</b> |
|                  | <b>Bibliography</b>                               | <b>30</b> |

# Chapter 1

## Introduction and Motivation

introduzione deve essere scritta alla fine

### 1.1 Project context (taken from **Methods\_GC\_LD.pdf**)

Thesis proposal, ecoinformatics, Lucia.

Eco-informatics is a modern way of conducting scientific research that enables scientists to generate new knowledge through innovative tools and approaches. Ecology is increasingly becoming a data-intensive science that involves acquiring, storing, and analysing large amounts of data from a range of sources, including satellite and aerial remote sensing, instruments, sensors, and human observations. Ecologists increasingly address large-scale questions with both scientific and social relevance, such as climate change and its future effects on ecosystems and the ecosystem services associated with them. The topic of this thesis project originated from a research study conducted by the Department of Earth, Environmental and Life Sciences (DISTAV, University of Genoa). The study analyses changes in the distribution of natural habitats in the Alps caused by future climate change in order to support decision-making on conservation planning and prioritisation. The analysis uses Species Distribution Models, a dataset of approximately four million records, and future climate-change scenarios based on General Circulation Models. The statistical software R (<https://www.r-project.org/>) is used to generate predictions of future changes in habitat distribution and biodiversity loss. The thesis student will improve the working script and its algorithmic functions to make the model analysis more efficient, drawing on their knowledge and studies in computer science. This thesis project involves interdisciplinary collaboration between DIBRIS and DISTAV, which can help produce better scientific results.

New draft for supervisor review

The proposal text above reports approximately four million records. The current working dataset contains 2,583,359 records; confirm the final value with the supervisors.

## 1.2 Thesis motivation

$$N_{\text{individual projections}} = S \times P \times R \times A \times (1 + F)$$

$$N_{\text{ensemble projections}} = S \times E \times (1 + F)$$

$$N_{\text{total projections}} = N_{\text{individual projections}} + N_{\text{ensemble projections}}$$

Here,  $S$  is the number of species,  $P$  the number of pseudo-absence repetitions,  $R$  the number of cross-validation repetitions,  $A$  the number of modelling algorithms,  $E$  the number of ensemble algorithms, and  $F$  the number of future scenarios. Each pseudo-absence dataset is split  $R$  times, and every algorithm is fitted to each resulting combination. Each fitted model is then used to make one raster map for the current environment and one raster map for every future scenario. The ensemble models are projected in the same way. A raster is a digital map divided into a regular grid. Each of the 63,951,097 cells represents one location and stores a numerical value. In a projection raster, that value is the prediction made by one fitted model for that location. A projection map applies one fitted model to all cells of one set of environmental rasters. The count therefore measures pairs of fitted models and environmental raster sets, rather than occurrence records or explicit iterations of the R loop. With the current F/F storage configuration, each pair is written as one raw prediction raster.

The production configuration uses ten pseudo-absence repetitions, five cross-validation repetitions and five modelling algorithms. Each species therefore has  $10 \times 5 \times 5 = 250$  individual models. Projecting them over the current environment and eight future scenarios gives  $250 \times (1 + 8) = 2,250$  individual projection maps per species. Across 167 species, this gives

$$167 \times 10 \times 5 \times 5 \times (1 + 8) = 375,750$$

individual projection maps. The two ensemble algorithms add  $167 \times 2 \times (1 + 8) = 3,006$  ensemble projection maps, for 378,756 model-projection combinations in total.

The current workflow models 167 Alpine grassland species from 2,583,359 presence records. Its environmental rasters contain 63,951,097 cells and five variables. For each species, the production configuration calibrates five algorithms and builds two ensemble models. It projects these models onto the current environment and eight future climate scenarios.

The workload combines a large spatial domain with repeated model calibration and raster projection.

The computational method determines whether the analysis can be completed at this scale. Historical runs lasted many hours. Several ended because of memory exhaustion, failed process forks, time limits, or file-writing collisions. Running the script on an HPC system does not solve these problems by itself. The software must control memory use, raster storage, and the scheduling of independent species.

This thesis examines the R workflow as HPC software. Leonardo provides the execution platform and the environment for measuring the program under different hardware allocations. CPU count, available RAM, storage policy, and filesystem activity affect runtime and completion. Our aim is to reduce execution time while preserving the scientific configuration, reproducibility, and output structure required by the ecological study.

## 1.3 Objectives and contribution

The scientific model and its parameters originate from the DISTAV study. This thesis contributes the software and experimental work required to execute that model at production scale. We adapted the R workflow to Leonardo and packaged its dependencies in a Singularity container. Each Slurm array task processes one species. We added runtime configuration, phase timings, resource measurements, success markers, and output checks. We also diagnosed memory, process-fork, and concurrent-write failures. We then compared worker counts and BIOMOD2 storage modes.

The first objective is to establish a reproducible baseline. Timings from runs with different inputs and parameters cannot provide such a baseline. The experiments identify the phases that dominate runtime. They also measure how worker count and storage settings affect elapsed time and memory. These measurements support later work on model parallelism and raster I/O. The thesis focuses on software performance. The ecological validity of the model choices and evaluation metrics remains under review with the DISTAV collaborators.

## 1.4 Preliminary evaluation method

The evaluation separates controlled benchmarks from historical and diagnostic runs. Controlled comparisons keep the species, input data, random seed, and scientific parameters fixed. Each comparison changes one execution setting. For every run, we record wall time, phase timings, CPU use, MaxRSS, completion status, and output validation. Failed, cancelled, and incomplete jobs show the resource limits of a configuration. We retain them as

evidence but do not use them as successful performance measurements.

End of new draft for supervisor review

# Chapter 2

## Background

real world and eventually related work

### 2.1 Ecoinformatics

### 2.2 Biological study context

An internal DISTAV methods document describes the biological study supported by the computational workflow. The study examines how climate change may alter the suitability of Alpine grassland habitats. It considers ten dry, alpine and subalpine habitat types from the EUNIS classification and 167 diagnostic plant species. Occurrence records are associated with environmental predictors at a spatial resolution of approximately 1 km. The predictors describe climate, soil and topographic heterogeneity. Species Distribution Models combine five modelling techniques and ensemble forecasts to estimate present and future habitat suitability.

The biological protocol also defines the analyses that follow model projection: changes in suitability, spatial shifts, habitat-level hotspot and cold-spot maps, and the relationship between suitability changes and species tolerance. This document provides the scientific context for the computing work, but it is not treated as the exact specification of every run. Dataset sizes, model repetitions and available climate scenarios changed during the project; the computational chapters therefore report the configuration recorded by the corresponding scripts and run manifests.

## **2.3 Data used**

## **2.4 Related work**

# Chapter 3

## Code as is

Existing workflow and computational problem.

### 3.1 R

BEGIN TODO: Container, workflow and Leonardo

The workflow received from DISTAV was a single R script written for a Windows workstation. It used absolute paths, loaded both **raster** and **terra**, and created a ten-worker **doParallel** cluster. The filename **ensamble.modelling.no.parallel.R** did not describe that version: model fitting and projection both used **nb.cpu = 10**. The script selected one species by a fixed index and limited that species to 100,000 occurrence rows. It also contained path-dependent parsing and a syntax error in the future ensemble call. The received version remains available in Git at commit **890f4c5**. The working copy has since been converted into a sequential reference by disabling the explicit cluster and setting every BIOMOD2 **nb.cpu** argument to one.

### 3.2 BIOMOD2

BIOMOD2 performs the scientific stages of the workflow. **BIOMOD·FormatingData** combines presence points, environmental rasters and random pseudo-absences. **BIOMOD·Modeling** calibrates GLM, GBM, ANN, FDA and MAXNET models under repeated cross-validation. **BIOMOD·EnsembleModeling** builds mean and coefficient-of-variation ensembles after filtering models with an evaluation threshold. The fitted models are then projected over the current environment and each future climate scenario. A final ensemble

forecast combines the corresponding individual projections.

The workflow handles large objects at several stages. Each environmental raster has 63,951,097 cells, and each projection uses five predictor layers. BIOMOD2 may run several model predictions at the same time and may retain or stack their raster outputs. Worker count and projection storage therefore affect the memory schedule even when the scientific parameters do not change.

### 3.3 Code pipeline

The original pipeline loads the occurrence table and all calibration rasters, selects one species, formats its data, calibrates the individual models and builds the ensembles. It writes model evaluations before projecting the individual and ensemble models over the current environment. A loop then loads each future climate directory, adds the static terrain and soil layers, and repeats both projection stages. The script records one duration for each main phase at the end.

All stages run from one long-lived R driver process, one working directory and a set of mutable objects. In the received version, this driver created temporary R worker processes through `doParallel` and BIOMOD2. Species selection, file paths, worker counts and scientific parameters were embedded in the script. There was no scheduler-level isolation between species and no completion marker that distinguished a finished analysis from a directory left by a failed run.

### 3.4 Original execution times

No controlled runtime is available for the unchanged DISTAV script. It cannot run in the repository environment without correcting its Windows paths and syntax, and the preliminary Spartaco records do not identify enough configuration details for a direct comparison.

### 3.5 Improved execution times

The measured Leonardo runs use adapted versions of the workflow. Chapter 5 reports those results and keeps them separate from the historical script described here.

END TODO: Container, workflow and Leonardo

### 3.6 Preliminary measurements on Spartaco

Before the Leonardo experiments, Lucia manually recorded four trial runs on Spartaco. The worksheet records per-phase time and RAM observations for two species, with different pseudo-absence settings and CPU counts. Table 3.1 preserves selected values from those records.

The measurements are useful for reconstructing the starting point of the project. They are not a controlled benchmark: the worksheet does not identify the code revision, container, hardware configuration, input version or measurement method. The source label `n_cells` is retained in the table because its meaning cannot be established from the worksheet alone. These results are therefore not compared directly with the later Leonardo measurements.

Table 3.1: Selected fields from four manually recorded trial runs on Spartaco. `n_cells` retains the source label. Time units are those recorded in the original worksheet; the maximum RAM is the largest phase value in that row.

| Species                    | <code>n_cells</code> | PA    | Rep. | RUN | CPU | Formatting | Models    | Future projection | Max RAM (GB) |
|----------------------------|----------------------|-------|------|-----|-----|------------|-----------|-------------------|--------------|
| <i>Omalotheca hoppeana</i> | 681                  | 10000 | 10   | 10  | 5   | 1.9 h      | 11.33 min | 3.65 min          | 52.9         |
| <i>Omalotheca hoppeana</i> | 681                  | 10000 | 10   | 10  | 10  | 2.02 h     | 7.74 min  | 8.07 min          | 56.16        |
| <i>Agrostis capillaris</i> | 100000               | 5000  | 5    | 5   | 5   | 32.25 min  | 53.87 min | 3.73 min          | 26           |
| <i>Agrostis capillaris</i> | 100000               | 10000 | 5    | 5   | 5   | 59.37 min  | 1.6 h     | 4.22 min          | 29.96        |

# Chapter 4

## My new code

Design and implementation of the improvements.

### 4.1 Container

BEGIN TODO: Container, workflow and Leonardo

We packaged the runtime in a Singularity image built from **rocker/geospatial**. The definition installs the required system libraries and the R packages used by BIOMOD2. It also creates the mount points used on CINECA and runs an installation check for the main R libraries. The production image contained BIOMOD2 4.3.4.5 and Terra 1.9.11.

The Slurm wrapper binds the repository root to **/work** inside the container and starts the R script from that path. Code, input paths and output paths are therefore the same inside each job. The image itself is not stored in Git because of its size. Copies are retained with the repository on Serviicola, Leonardo and Spartaco.

The current definition uses the mutable base tag **rocker/geospatial:latest**. A future rebuild should pin the base image or record its digest before the container is treated as independently reproducible.

### 4.2 Parallel model execution

The adapted workflow assigns one species to each Slurm array task. The array index selects the species, and every task writes to an isolated species directory. This design uses separate nodes for independent species without splitting the occurrence records of

one species or changing its fitted models. The production configuration uses ten pseudo-absence repetitions with 10,000 pseudo-absences each, five cross-validation repetitions and a 70% calibration split.

Within a task, `SLURM_CPUS_PER_TASK` directly determines the BIOMOD2 model and projection worker count. Ensemble stages use at most two workers, and BLAS and OpenMP thread counts are fixed at one to avoid nested parallelism. BIOMOD2 registers the required backend, so the external `doParallel` cluster from the historical script is no longer created.

The projection settings are also explicit. Both `keep.in.memory` and `do.stack` are set to `FALSE`. BIOMOD2 then writes individual model projections separately and does not retain a combined raster stack in the returned object. Ensemble forecasting receives the projection object through `bm.proj`, so it reuses completed projections instead of starting the same individual projection again. The worker removes the target species directory before a fresh run and writes a `_SUCCESS` marker only after all current and future stages finish.

## 4.3 Parallel I/O

The current implementation does not provide a separate parallel I/O algorithm. BIOMOD2 workers may write model rasters concurrently, while `do.stack = FALSE` changes the representation and lifetime of those outputs. This is part of the measured storage configuration, not evidence of an independently optimized I/O stage.

Large inputs, model outputs and the Singularity image remain outside Git. The command `make omni-sync` invokes `scripts/omni-sync.sh` and copies either selected relative paths or, by default, `data/` and the container from Leonardo to matching paths on Spar-taco. Serviicola acts as the transfer relay without storing an intermediate data copy. The script uses `rsync` first and falls back to `scp -3`; neither path deletes destination files.

Transfer duration, throughput and filesystem contention were not measured. No performance claim is made for the transfer workflow, and the intended scope of parallel I/O still requires a separate definition and benchmark.

END TODO: Container, workflow and Leonardo

# Chapter 5

## Experimental results

Experimental methodology and results.

New draft for supervisor review

Convert the approved results to numbered LaTeX tables after supervisor review.

### 5.1 Leonardo

The experiments ran on Leonardo DCGP nodes. Each node provides 112 CPU cores and 494,000 MB of allocatable memory. A Slurm array task processed one species on one node. The full-raster runs used five environmental variables over 63,951,097 cells. They produced projections for the current environment and eight future scenarios. Most experiments requested all allocatable node memory. This reserved the whole node for accounting purposes, even when the R process used fewer CPU cores.

We checked the coordinate reference system inside the production container on Leonardo. All 18 environmental rasters reported WGS 84 (EPSG:4326) and had the same geometry. The occurrence coordinates ranged from  $-24.0043$  to  $66.1957$  degrees longitude and from  $29.5124$  to  $71.1540$  degrees latitude. They fell within the raster extent and aligned with its grid-cell centres within the precision of the CSV input. No reprojection was required.

We evaluated completion separately from resource use. A zero process exit code was not sufficient when Slurm recorded an OOM event. A Slurm failure state also did not identify the failing R phase by itself. A run counted as complete only when its logs reached the end of the workflow. The expected timing files, output directories, non-empty files, and success marker also had to be present.

## 5.2 Tests

The first validated full-raster reference used the experimental E5 configuration, with five pseudo-absence repetitions, and four BIOMOD2 workers in F/F storage mode. Both projection storage options were disabled. The run completed in 22 hours, 33 minutes, and 45 seconds. Slurm reported a MaxRSS of 260.94 GiB. The species directory contained 1,364 non-empty files and occupied 7.18 GiB. Four additional summary files were written at the output root. The outputs included 125 individual models, two ensemble models, and all eight future scenarios.

The F/T configurations used stacked projections and failed during the first future scenario. The two-worker run stopped after 5 hours and 44 minutes. The four-worker run stopped after 3 hours and 57 minutes. In both cases, `mcfork()` could not allocate memory. Their Slurm MaxRSS values were 413.22 and 413.80 GiB.

Among the validated F/F runs, elapsed time was lower at higher worker counts. These counts apply to individual modelling and projection; all ensemble phases were capped at two workers. Six individual-model and projection workers completed in 17.85 hours, with an observed wall-time ratio of 1.264 relative to the four-worker reference. Eight completed in 15.09 hours, with a ratio of 1.495. Sixteen completed in 10 hours and 21 minutes, with a ratio of 2.179. The accounting, timing files, output structure, and success marker of the 16-worker run were validated. The exact code revision of the four-worker reference was not recorded, so these ratios describe the observed runs but do not isolate the effect of worker count.

Slurm MaxRSS was 258.95 GiB with six workers and 278.67 GiB with eight workers. It rose to 464.77 GiB with sixteen workers. CPU allocation efficiency fell from 68.1% to 60.0% and then 45.1%. A 32-worker F/F run reached 481.16 GiB. It was killed during the current projection after 1 hour and 40 minutes.

The complete future loop accounted for 77.4% to 83.5% of wall time in the validated runs. Formatting remained between 4,485 and 4,601 seconds and did not use the configurable model-worker count. The current ensemble projection remained between 1,422 and 1,517 seconds while its worker count stayed fixed at two. Among the F/F runs with separate scenario records, future individual projections fell from 41,093 seconds with six workers to 17,227 seconds with sixteen. The corresponding future ensemble totals remained close to 11,630 seconds because those calls stayed at two workers. The four-worker F/F reference retained only the combined future time, so it cannot be used for this split comparison.

A separate controlled benchmark compared split and monolithic R implementations on the same node. Five runs of each version used a fixed seed, one worker, a small input, and no future projections. The split version averaged 26 minutes and 10 seconds with 47.09 GiB MaxRSS. The monolithic version averaged 26 minutes and 28 seconds with 47.63 GiB

MaxRSS. The observed differences were 18 seconds and 0.54 GiB in favour of the split version. These results do not establish equivalence between the implementations. They also do not establish effects on parallelism or reproducibility.

The production configuration uses ten pseudo-absence repetitions, 10,000 pseudo-absences per repetition, five cross-validation repetitions and a 70% calibration split. Three species completed with this configuration. *Achillea atrata* took 25 hours, 36 minutes, and 36 seconds. *Achillea chusiana* took 21 hours, 26 minutes, and 42 seconds. *Agrostis capillaris* took 33 hours, 58 minutes, and 34 seconds. Their Slurm MaxRSS values were 316.16, 315.78, and 291.42 GiB, respectively.

Each species directory contained 2,614 non-empty files and a success marker. Four additional summaries were present for each run. The directories occupied 13.98, 13.36, and 59.13 GiB. In the recovered phase timings, the future loop ranged from 59,841 to 89,815 seconds. Individual modelling ranged from 172 to 10,989 seconds. This configuration performs more model runs than the earlier worker-count experiment, so their timings must not be compared directly.

### 5.3 Discussion and lessons learned

The validated F/F runs had lower full-raster runtimes at higher worker counts. Since the exact code revision of the four-worker reference was not recorded, this comparison does not isolate the effect of worker count. Memory use rose sharply at sixteen workers, and the 32-worker run did not complete. The successful four-worker T/F run was 0.9% faster than the F/F reference and used 3.75 GiB more peak memory. Each storage mode was run once, so the comparison does not establish a storage effect. The F/T configurations, which enabled stacked projections, failed before the first future projection could complete.

Failed runs remain part of the evaluation. A failed run can reach its stopping point sooner when given more workers. This shorter time is not a speedup because the run produces no valid result. Runtime comparisons must include completion status, MaxRSS, and output checks. They must also keep scientific parameters fixed. The production runs with ten pseudo-absence repetitions answer a different performance question from the earlier runs with five.

Resource allocation affected both cost and runtime. The three completed production species consumed about 9,076 CINECA local hours. Each task reserved all allocatable node memory. The replacement array for species 4 to 167 did not start R. CINECA cancelled it in the administrative prolog after the project exceeded its budget. Future campaigns need a memory request based on measured MaxRSS, with a margin for variation between species. They should not request the full node unconditionally.

End of new draft for supervisor review

BEGIN TODO: Container, workflow and Leonardo

## 5.4 Operational difficulties on Leonardo

The summary above reports the successful measurements. This section records the failure paths that led to the final workflow and distinguishes resolved execution problems from open scientific and operational work.

### 5.4.1 Projection memory and process forks

The first production-scale array used BIOMOD2's default in-memory stacked projections. Its first five species tasks ended in out-of-memory conditions after 13 to 17 hours. Later F/T tests disabled retained values but still built stacks. They stopped while creating the clamping mask for the first future scenario because `mcfork()` could not allocate another worker process. These failures showed that a large memory request did not make the stacked representation safe.

The mitigation was to set both projection storage options to **FALSE** and to vary the worker count without changing the node-memory request. This configuration completed with 4, 6, 8 and 16 workers. It still used substantial memory: the 16-worker run reached 464.77 GiB, and the 32-worker run ended in OOM during current projection. F/F storage is therefore the validated representation for these runs, but it does not remove the need for a measured worker and memory limit.

Code inspection also found a MAXNET-specific risk. The other four algorithms delegate raster prediction to Terra, which can process blocks. The MAXNET path converts the complete raster to points and then to a data frame. A local blockwise prototype produced identical values on 226,775 valid cells, but it has not been validated on full rasters, all models or all scenarios. It remains outside the production workflow.

### 5.4.2 Concurrent writes and stale output

Several failures were unrelated to total memory. Job **48325677\_3** reported that a projection file already existed while parallel workers wrote the same output. Jobs **48418427** [3-5] failed on pre-existing outputs, while other runs recorded invalid paths or filenames.

The adapted workflow uses isolated output directories, removes the selected species direc-

tory before a fresh run and passes **bm.proj** to ensemble forecasting. Later validated runs contained no empty files and showed the expected current and future structure. Automatic resume from partial output is not enabled because a stale file may belong to a different model or configuration.

### 5.4.3 Long phases without visible progress

The pseudo-absence step initially appeared to be stalled. Process accounting showed that R was still using approximately one CPU core while BIOMOD2 repeatedly scanned the 64-million-cell raster. The production choice of ten pseudo-absence repetitions is a scientific setting and was not reduced as a performance workaround.

We added timestamps, per-phase measurements and one timing record for each future scenario. These records distinguish slow computation from a stopped process and identify the phase reached before a failure. Formatting and MAXNET prediction remain candidates for later optimization, but neither has a production replacement validated across the complete workflow.

### 5.4.4 Scheduling, maintenance and time limits

Scheduling was constrained by the long-production QOS and by a project limit of three concurrent nodes. Maintenance reservations prevented jobs whose requested wall time overlapped the outage from starting. Two 12-worker tests were then stopped after 9 hours and 40 minutes when maintenance began. Their elapsed times are imposed limits, not complete runtimes.

Shortening the requested limit allowed some tests to run before maintenance, but this was an operational scheduling change rather than a code optimization. The incomplete 12-worker jobs are retained in the run inventory and excluded from successful runtime comparisons.

### 5.4.5 Accounting and budget exhaustion

Most experiments requested all 494,000 MB allocatable on a DCGP node. CINECA accounting charged the resulting node allocation even when BIOMOD2 used only a small fraction of its 112 cores. By the time the first three production species had completed, cumulative project accounting had reached 103.4% of the 100,000 local-hour budget.

The replacement array for species 4 to 167 received nodes for only 1 to 16 seconds per task. The controller cancelled those tasks in the administrative prolog with **insufficient**

**or expired budget.** Singularity, R and the batch script did not start, so the zero application exit status did not indicate a completed analysis. A new campaign requires additional budget and a memory request derived from measured MaxRSS with a margin for variation between species.

#### 5.4.6 Completion and scientific validation

Slurm state, application exit code and scientific validity are separate checks. Some OOM jobs retained a zero application exit code, and a **\_SUCCESS** marker only proves that the R script reached its final write step. We therefore combined Slurm accounting, phase logs, non-empty file counts, output structure, timing tables and the success marker when classifying a run as complete.

This procedure validated the 16-worker F/F run and the three completed production species. It does not resolve the GLM, MAXNET, integer-overflow and missing-metric warnings recorded in successful logs. Those warnings remain open scientific-review items and prevent the execution checks from being treated as validation of the ecological results.

**END TODO: Container, workflow and Leonardo**

# Chapter 6

## Conclusions and future development

### 6.1 Conclusions

### 6.2 Future development

### 6.3 Ecoinformatics

What it is.

Why large datasets are used.

Why high-performance computers are used.

Purpose.

### 6.4 Data used

In the **Eval...** matrix, results with **calibration** below 0.5 are considered less performant.

### 6.5 Computer science objective

To be defined.

## 6.6 Interdisciplinary collaboration

Optional section.

# Chapter 7

# Appendix

Temporary or optional chapter.

# Bibliography

- [CHE] CHELSA. CHELSA climatologies. Dataset documentation; accessed 2026-09-01. URL: <https://www.chelsa-climate.org/datasets/chelsa-climatologies>.
- [CINa] CINECA. Access to the systems. HPC documentation; accessed 2026-09-01. URL: <https://docs.hpc.cineca.it/general/access.html>.
- [CINb] CINECA. Leonardo. HPC documentation; accessed 2026-09-01. URL: <https://docs.hpc.cineca.it/hpc/leonardo.html>.
- [CINc] CINECA. Singularity and apptainer containers. HPC documentation; accessed 2026-09-01. URL: <https://docs.hpc.cineca.it/services/singularity.html>.
- [GBLPT26] Maya Guéguen, Hélène Blancheteau, Rémi Lemaire-Patin, and Wilfried Thuiller. *biomod2: Ensemble Platform for Species Distribution Modeling*, 2026. R package version 4.3-4-5. URL: <https://CRAN.R-project.org/package=biomod2>, doi:10.32614/CRAN.package.biomod2.
- [HBB26] Robert J. Hijmans, Andrew Brown, and Márcia Barbosa. *terra: Spatial Data Analysis*, 2026. R package version 1.9-11. URL: <https://rspatial.org/>.
- [KCB<sup>+</sup>17] Dirk N. Karger, Olaf Conrad, Jörg Böhner, Tobias Kawohl, Holger Kreft, Ruben W. Soria-Auza, Niklaus E. Zimmermann, H. Peter Linder, and Michael Kessler. Climatologies at high resolution for the earth’s land surface areas. *Scientific Data*, 4:170122, 2017. doi:10.1038/sdata.2017.122.
- [R C26] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2026. URL: <https://www.R-project.org/>.
- [TCA24] Matteo Turisini, Mirko Cestari, and Giorgio Amati. LEONARDO: A pan-european pre-exascale supercomputer for HPC and AI applications. *Jour-*

*Journal of Large-Scale Research Facilities*, 8:A186, 2024. doi:10.17815/  
jlsrf-8-186.